Advances in Cognitive Systems X (20XX) 1-6 Submitted X/20XX; published X/20XX

Formalization of Task-Method-Knowledge Representations for Automatic Verification of Agent Self-Models

Ashok K. Goel ASHOK.GOEL@CC.GATECH.EDU
Jeanette Luu JEANETTE.LUU@GATECH.EDU
Nicholas E. Alico NALICO3@GATECH.EDU
Shubham Puri SPURI62@GATECH.EDU
Bukhaar A. M. Mahamed BMAHAMED3@GATECH.EDU
Spencer Rugaber SPENCER@CC.GATECH.EDU
Georgia Institute of Technology, Atlanta, GA 30332 USA

Abstract

Self-Explanation refers to an AI agent explaining its reasoning and justifying its results to a human user. One way to enable a knowledge-based AI agent to explain itself is by endowing the agent with a self-model of its knowledge and reasoning, and by adding a metacognitive layer that introspects on the self-model to generate explanations. We express an agent's self-model in the form of a Task-Method-Knowledge (TMK) representation that captures how the methods and the knowledge of the agent accomplish its tasks. This raises the issue of automating the verification of the TMK model. We present a formalization of the TMK knowledge-representation language in Extended Backus–Naur Form (EBNF)[1] that enables partial automation of the verification of the agent self-model.

1. Introduction

Self-Explanation in AI is a significant research topic that pertains to an AI agent explaining its reasoning and justifying its results to a human user (Swartout, 1983). One way to enable a Knowledge Based AI (KBAI) agent [2] to explain itself is by endowing the agent with a self-model of its knowledge and reasoning, as well as adding a metacognitive layer to the agent's architecture that introspects on the self-model to generate self-explanations (Cox & Raja, 2011; Goel et al., 1996).

This immediately raises two interrelated issues: What kinds of explanations do human users seek of KBAI agents, and what kinds of self-models of AI agents may generate the needed

[1] . https://en.wikipedia.org/wiki/Extended_Backus-Naur_form

[2] . This article pertains to self-explanation in knowledge-based AI agents. Although it uses generative AI as a method to generate self-explanations in KBAI agents, it does not cover self-explanation in generative AI agents.

© 20XX Cognitive Systems Foundation. All rights reserved.

explanations? In prior work on participatory design of an AI agent called Skillsync that matches employee skills acquired in higher education to available jobs and employer needs (Robson et al., 2022), we found that both company and college users were interested in understanding how the AI agent worked to produce its recommendations (Lee et al., 2023). Many of the questions asked by company and college users sought explanations about the vocabulary, the knowledge, and the reasoning used by the agent; questions about reasoning pertained to the context, the processing, and the goal of the agent's reasoning. We generalized these findings into three main categories of questions: *What? Why?* and *How?* Briefly, *What?* questions pertain to explanations about the vocabulary and knowledge used by the agent; *Why*? questions refer to explanations about the tasks or the goals of the agent; and *How?* questions relate to explanations about the reasoning methods of the agent.

In the Skillsync project, we used agent self-models in the form of Task-Method-Knowledge (TMK) representations that capture how the methods and the knowledge of the agent accomplish its tasks (Murdock & Goel, 2008; Goel & Rugaber, 2017). The TMK model of an AI agent is organized in a T -> M -> T -> M . . . -> T -> K hierarchy. In a variety of domains ranging from building social connections among students in an online class (Basappa et al., 2024) to inquiry-based modeling of ecological systems in an interactive learning environment (Sushri et al., 2024; Goel et al., 2026), we showed that introspection over TMK self-models of AI agents helps answer *What?*, *Why?* and *How?* . Briefly, the K(knowledge) component of a TMK model addresses *What?* questions; the T(ask) component addresses *Why?* questions; and the M(ethod) component is intended to answer *How?* questions. However, since the TMK model is organized in a T -> M -> T -> M . . . -> T -> K hierarchy, the answer to a *How?* question may encompass both methods and (sub)tasks.

In the current state of the art, knowledge engineers typically handcraft the TMK models of AI agents in the TMKL2 knowledge representation language (Rugaber, 2011). The process of handcrafting a TMK model of an AI agent includes manual construction of a preliminary TMK model by a knowledge engineer, verification of the model by visual inspection by one or more additional knowledge engineers, validation through question-asking by a human user, revision and refinement of the TMK model, and so on. Here, verification focuses on building the model "right" and validation on building the "right" model (Boehm, 1981). This iterative process is expertise demanding, time consuming, labor intensive, and error prone. This adversely impacts the quality of explanations and limits the consistency, generality and scalability of TMK models of AI agents.

The question then becomes: how can we automate this elaborate process of model construction? Even partial automation of the model construction process would represent progress towards the goals of consistency, generality, and scalability (Schmidt, 2006). Thus, we have formalized the syntax of the TMKL2 grammar using the Extended Backus-Naur Form (Backus, 1959; Backus et al., 1963; Scowen, 1996; Wikipedia, 2026) to partially automate model verification in the process of model construction. We chose EBNF for the formalization of the TMKL2 grammar because EBNF is both machine- and human-readable; in fact, it is designed to be so. We also built a TMK Modeler Platform that includes a graphical user interface (GUI), a syntax parser and a static semantic analyzer for the TMK models expressed in the EBNF TMKL2 grammar. The platform GUI automatically builds the abstract syntax tree (AST) for the TMK model of the agent for inspection by knowledge engineers. This formalization of the TMKL2 knowledge representation grammar helps partially automate model verification in two ways. First, the parser automatically verifies the syntax of the TMK model: Does the model follow the rules of the TMKL2 grammar? Second, the AST generated by the platform GUI allows the knowledge engineer to more accurately, easily, and

efficiently inspect the structure of the TMK model: For example, does the nesting of the TMK model follow the T -> M -> T -> M . . . -> T -> K schema?

In this paper, we present the formalization of the TMKL2 grammar in EBNF, a TMK Modeler Platform, a metacognitive architecture for self-explanation, and an algorithm that uses Generative AI for generating self-explanations derived from the TMK models of AI agents. Although the TMLK2 language, the TMK Modeler Platform, the metacognitive architecture, and the algorithm for self-explanation are general-purpose and domain-agnostic, below we use self-explanation in the Virtual Ecology Research Assistant, VERA, an interactive learning environment for inquiry-based modeling (An et al., 2020, 2025), for illustration.

2. Related Work

Our work connects with AI research work on metacognition, self-explanation, agent models, and integration of Knowledge-Based AI (KBAI) and generative AI (GenAI) methods.

2.1 Metacognition

Metacognition in AI agents has a prolific history. The Cox & Raja (2011) anthology, for example, positions metacognition as applicable to many AI tasks such as monitoring, control, introspection, reflection, self-awareness, and self-explanation. They delineate several levels of processing in metacognition in AI agents:

1. Ground-level, where percepts in the environment are directly mapped into actions on it.
2. Deliberative level, at which world models help interpret the percepts and form plans that specify the actions.
3. Meta-level, at which a model of the deliberation process helps with monitoring, control, introspection, etc.

In many metacognitive architectures such as MIDCA (Cox et al., 2016), the meta-level contains a model of the deliberative- and ground-level processing of the agent. Our work on generalizable, reproducible, and transferable self-explanation capabilities in AI agents adopts a similar stance: it endows a metacognitive layer in an agent's architecture with a formalized TMK model of agent itself.

2.2 Self-Explanation

As AI agents become more complex and prevalent, there is a growing need to design agents responsibly, which includes making their inner working transparent. In AI in education, for example, transparency of AI agents helps improve trust in its reasoning and results (Basappa et al., 2024). One way to provide transparency in AI agents is to enable the agents with the capacity for self-explanation, the process by which agents articulate the thinking behind their choices or conclusions. When users can interact with an agent and inquire about how they reach a particular conclusion or recommendation, it opens opportunities for reflection, alignment with their internal goals, and avenues for deeper learning. Learners who can access meaningful, context-aware explanations from AI agents are more likely to engage cognitively, retain knowledge, and develop confidence in the learning agent (Gajos & Mamykina, 2022).

2.3 Agent Models

The origins of TMK self-models of AI agents lie in early research on knowledge systems. Chandrasekara et al. (1989) described a scheme for explanation of KBAI agents in terms of tasks and methods. Chandrasekaran & Swartout, (1991) proposed explicit representation of the design of KBAI agents for enabling explanation of their reasoning. TMK models go a step further: a TMK model of an agent views the agent as an "abstract device" with causality and teleology, and explicitly specifies the design of the device in terms of the tasks, methods, and knowledge used by the agent (Murdock & Goel, 2008).

TMK models are like Hierarchical Task Networks (HTN) ((Erol et al., 1994; Nau et al., 2003) in classical AI research on planning. Hoang et al., (2005) have shown that while HTN models can be made to capture the same knowledge as TMK models, the TMK representations are more expressive than the HTN representations: TMK models capture the expectations of tasks and subtasks which is useful for both prediction and explanation. More importantly, TMK models capture conceptual and relational knowledge of the domain that is critical for meaningful explanations.

2.4 Combining Knowledge-based AI and Generative AI

In the Skillsync project mentioned above, we used the TMKL2 knowledge representation language for building TMK models of the agents and systematic search over the models to generate self-explanations (without any use of generative AI methods). In more recent work, we have combined KBAI and GenAI approaches for generating self-explanations. For example, SAMI is an AI agent for connecting learners with one another in large online classes (Kakar et al., 2024). SAMI integrates KBAI and GenAI for generating self-explanations (Basappa et al. 2024): KBAI provides knowledge structures such as TMK models and GenAI searches over the knowledge structures to generate self-explanations and answer a user's questions. Table 1 summarizes the empirical results on self-explanation from the SAMI experiments.

Table 1. Summary of empirical results on self-explanation from the SAMI experiments

Evaluation	SAMI
Dataset	66 questions from the XAI question bank (Liao et al., 2020) adapted for SAMI
Metrics	Correctness and Completeness
Correctness result	49/66 correct, 8/66 partially correct; 9/66 incorrect
Completeness result	48/66 complete, 18/66 incomplete

3. TMK Model of a Knowledge-based Agent

3.1 Knowledge-based agent: VERA

VERA is an interactive learning environment to support learners in thinking like scientists through inquiry-based learning (An et al., 2020, 2025). It enables learners to create conceptual models and run agent-based simulations of ecological systems to test their hypotheses of a given phenomenon. Figure 1 illustrates the flow of control and data in VERA. A learner starts with a given ecological phenomenon (shown on the left side of the figure) such as the invasion of the Kudzu vine in southeast USA that is killing the American Hornbeam tree. The learner builds a conceptual model of the ecological system using the model editor (shown in the center), sets the values for the parameters of the model (such as the rate of growth of Kudzu and American Hornbeam), and then VERA uses the NetLogo[3] platform to simulate the conceptual model and make predictions about the populations of Kudzu and American Hornbeam over time (shown on the right). A learner may conduct what-if experiments by changing the parameter values or revising the model itself. Thus, VERA acts as a virtual laboratory for fostering inquiry-based learning.

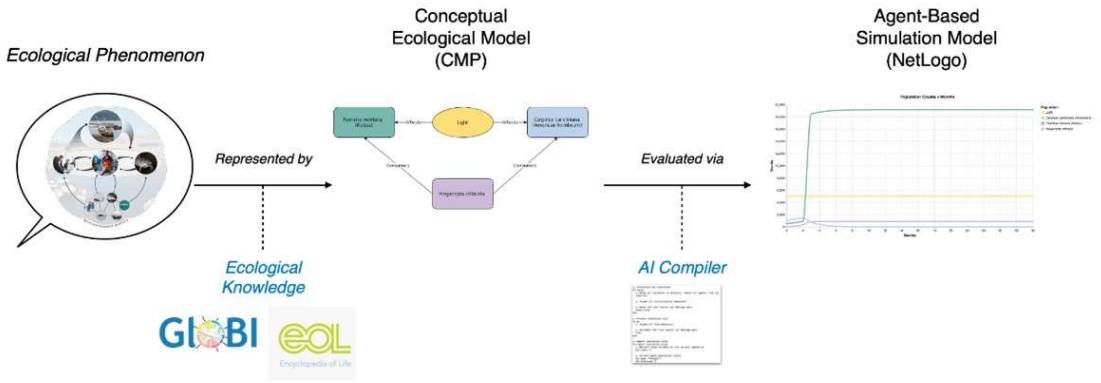


Figure 1. The flow of data in VERA from a given ecological phenomenon on the left to a conceptual model of the phenomenon in the middle to the results of the agent-based simulation on the right.

3.2 TMK model of VERA

Figure 2 provides a segment of a hand-crafted TMK model of VERA. The top-level goal of VERA is to support the learning of ecological concepts by introductory biology students. The TMK model of VERA decomposes the top-level goal into sub-level goals: 1) to enable learners to conduct an experiment in ecology by facilitating their construction of a conceptual model of an ecological system, 2) setting values for the model parameters, and 3)running simulations of the model.

[3] . https://www.netlogo.org/

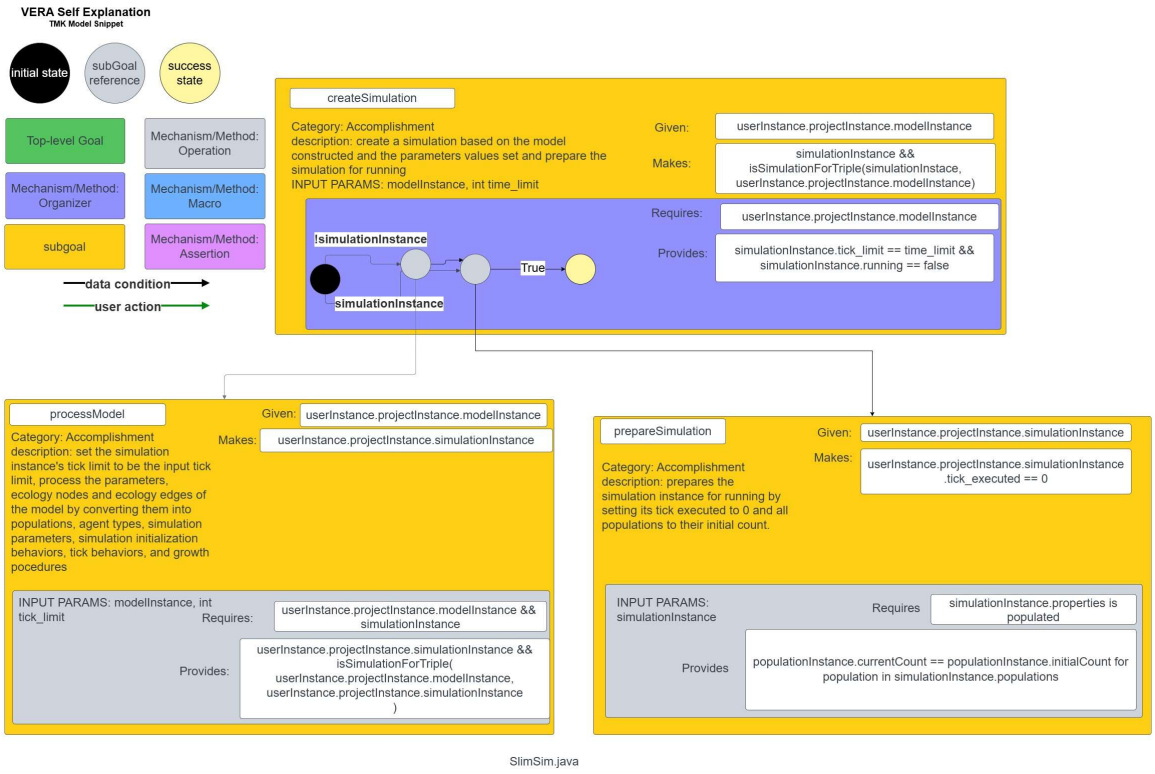


Figure 2. A segment of the TMK model of VERA. A legend is situated on the top-left. This portion of the much larger VERA TMK model identifies some of the goals, methods, states, transitions, and outputs in the TMK knowledge representation.

The portion of the TMK model shown in Figure 2 highlights how one of the decomposed sublevel goals of creating a simulation is further distilled into two smaller sub goals: processModel and prepareSimulation, as well as how to accomplish those sub goals by checking different input parameter(s), data conditions, and transition states.

Tables 2, 3, and 4 depict the *Task*, *Method*, and *Knowledge* TMKL2 language constructs, respectively, and the formalized grammar in EBNF to represent them.

We use the TMK models of VERA to illustrate how the models represent knowledge. For example, the goal of creating a simulation in Figure 2 is modeled as a *Task*, with key–value pairs that capture information such as *name*, *description*, *input/output parameters*, preconditions (*givens* the user created a project and that project has a model), post-conditions (*makes* an instance of simulation, set the instance properties, and prepare the instance for running), and *means* (mechanism) to accomplish the createSimulation goal.

The mechanism to accomplish a *Task* is modeled as a *Method* which has its own key–value pairs that capture *name*, initial state, precondition (*requires* data conditions), post-conditions (*provides* output), and state transitions. For example, to accomplish the createSimulation goal, the preconditions expect a user-created project, that the project has a model, and that a simulation has not already been created for said model. The outputs are a simulation instance, and transition

Table 2. Task Schema

Element	Description	EBNF Grammar
Name	An identifying string.	taskName = identifier ;
Description	An optional free-form textual string.	taskDescription = stringLiteral | "" ;
Input parameters	A sequence of FormalArguments, each of which is a pair <Name, Type>.	inputParameters = {formalArgument};
Output parameters	A sequence of FormalArguments.	outputParameters = {formalArgument} ;
Given	An optional LogicalExpression. If a Given condition is not explicitly provided, the default value is true.	given=logicalExpression | True ;
Makes	An optional LogicalExpression. If a Makes condition is not explicitly provided, the default value is true.	make=logicalExpression | True ;
Means	Description of the method of how an agent accomplishes its task.	means = {(mechanismReference, actualArgument)+} ;

Table 3. Method Schema

Element	Description	EBNF Grammar
Name	An identifying string.	mechanismName = identifier;
Description	An optional free-form textual string.	mechanismDescription = stringLiteral | "" ;
Input parameters	A sequence of FormalArguments, each of which is a pair <Name, Type>.	inputParameters = {formalArgument} ;
Output parameters	A sequence of FormalArguments.	outputParameters = {formalArgument} ;
Requires	An optional LogicalExpression. If a Given condition is not explicitly provided, the default value is true.	requires = logicalExpression | True ;
Provides	An optional LogicalExpression. If a Makes condition is not explicitly provided, the default value is true.	provides = logicalExpression | True ;

Table 4. Knowledge Schema

Element	Description	EBNF Grammar
Name	An identifying string.	conceptName = identifier ;
Description	An optional free-form textual string.	conceptDescription = stringLiteral | "" ;
SuperConcept	Either zero or more ConceptReferences to parent Concepts or one of two special designators, ENUMERATED and EXTERNAL.	superConcept = conceptReference | ENUMERATED | EXTERNAL ;
Properties	A sequence of Properties, each of which comprises a Name, a Type and an optional DefaultValue of the corresponding Type.	properties = property+ ;

from the initial *source state*, "simulation_instance", to the *target state*, "simulation_instance_created". Then from the *source state*, "simulation_instance_created", it transitions to the next *target state*, "prepare_simulation".

The above target state, "prepare_simulation", is also modeled as a *Method* with its own requirements (data conditions), "simulation_instance_created", and provides (output), "simulation_instance" with parameters reset to default values.

The specification of a method captures causal relationships among the states, where the data conditions of a preceding state enable a transition to occur, and the specification of the transition expresses the succeeding state. Together, the specifications of the task and the method capture the teleology of the agent viewed as an abstract device.

4. **Generating Self-Explanations**

4.1 Data flow Architecture of the Metacognitive Layer

Figure 3 illustrates the data flow architecture of the metacognitive layer in VERA. The Metacognitive Module (MCM) contains a TMK model of VERA. When a user enters their question to VERA via a web browser, VERA sends the question to the metacognitive layer (called MCM-TMK in Figure 3) via an API call. We use the FAISS (Johnson et al., 2021) library which allows us to search the embeddings of TMK documents that are similar to each other. For similarity search, a user's question is combined with a prompt context string and the FAISS vector store of TMK documents to create a LangChain (Chase, 2022) PromptTemplate. MCM-TMK pre-loads TMK models in its working memory as key-value pair dictionaries when the MCM-TMK server starts. For every question, MCM-TMK searches for the keys in all dictionaries by semantically matching the question with the keys. Then, it uses the top *k* documents with the highest similarity scores and composes the information together to form its answer.

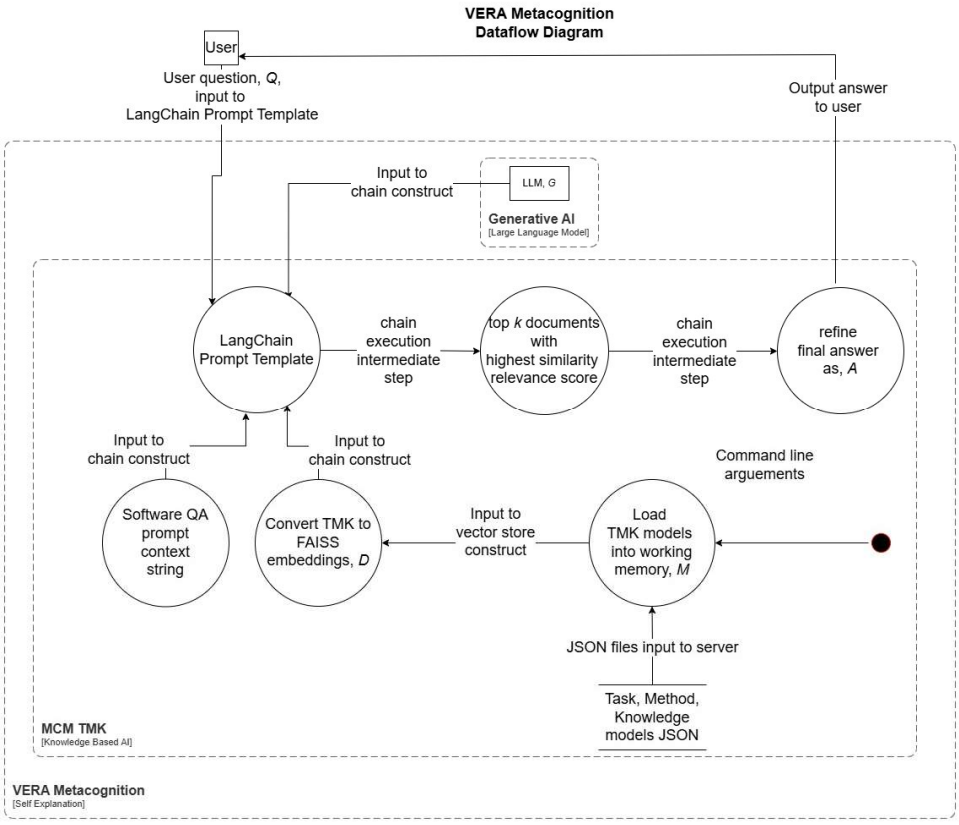


Figure 3. The Dataflow Architecture for Metacognition in VERA. A user's question is combined with a prompt context string and the FAISS vector store of TMK documents to create a LangChain PromptTemplate. The top *k* most relevant TMK documents are selected and fed to the LLM to generate an initial answer. After iteratively refining the answer contents, a final answer is returned to the user.

For example, a user may ask the question: "How does VERA create a simulation?" MCM-TMK first searches and matches the user question with the key, createSimulation, in *Task* model. Then, MCM-TMK searches the *Method* model again using the key createsSimulation and finds the next key, simulation_instance_created, which is the output of the method createSimulation. Using this key, simulation_instance_created, to search further, MCM-TMK would find both methods prepareSimulation and processMmodel require simulation_instance_created as a data condition. Now we have the top *k*, (*k* = 4 in this case), documents for the language model to piece together the information to form the answer.

4.2 Algorithm for Generating Self-Explanations in VERA

Algorithm 1 describes how the TMK model enables generative AI (OpenAI's gpt-3.5-turbo-1106 model used in this example) to generate self-explanations.

Algorithm 1 VERA Self-Explanation

Input: User question Q, TMK model M (Task/Method/Knowledge), LLM G, bounds k
Output: Explanation A

1: [VERA] $C \leftarrow$ SELECTCORPUS(Q,M) ▷ M.Task ∪ M.Method ∪ M.Knowledge 2: [VERA] $D \leftarrow$ PREPROCESS(C) ▷ normalize, deduplicate, chunk
3: [VERA] $E_{\text{docs}} \leftarrow$ EMBED(D); $e_Q \leftarrow$ EMBED(Q)
4: [VERA] $S \leftarrow$ TOPK(e_Q,E_{docs},k) ▷ FAISS similarity search
5: [VERA] $A \leftarrow \epsilon$ ▷ empty string
6: for all $d \in S$ (in ranked order) do
7: if $A = \epsilon$ then
8: [GPT] $A \leftarrow G$("Answer using",{Q,d})
9: else
10: [GPT] $A \leftarrow G$("Refine prior answer",{Q,A,d})
11: end if
12: end for
13: [GPT] $A \leftarrow G$("Apply personal style/POV to",A)
14: [VERA] return A ▷ optional refinement

- Corpus Selection (L1): A relevant corpus (C) is selected based on the query, utilizing M's *Method*, *Knowledge*, or a combination including *Task*.
- Preprocessing and Embedding (L2–L3): (Line 2) normalizes, deduplicates, and chunks the corpus. The resulting chunks (D) and the query (Q) are then converted into vector embeddings (E_{docs} and e_Q) (Line 3).
- Document Retrieval (L4–L5): Using vector similarity search, the system retrieves the top-k documents: The answer (A) is initialized as an empty string (ϵ) (Line 5).
- Iterative Refinement Loop (L6–L12): The system iterates through each retrieved document $d \in S$ in ranked order.
  - Initial Answer (L7–L8): If $A = \epsilon$, the LLM generates an initial answer grounded in the current document d:
  - Refinement (L9–L10): If $A \neq \epsilon$, the LLM refines the existing answer by incorporating new evidence from document d:
- Optional Refinement and Return (L13–L14): The LLM performs an optional final refinement to apply the user's personal style and/or point of view (POV): The final explanation (A) is then returned.

5. The Making of a Formalized TMK model

Our initial approach to construct a TMK model was manual. We handcrafted a pictorial of TMK model such as Figure 2 first, then manually translated the pictorial TMK model to a text

representation in JSON format. The structure of each TMK model of a specific agent was entirely based on the expertise of the TMK modeler. This resulted in the verification of the TMK models of specific agents also being manual, time-consuming, error-prone, and dependent upon human subject matter expertise. For a TMK model to be automatically verifiable, we need rules, specifically syntax rules or grammar for the TMKL2 language. We use EBNF to define the syntax rules. The grammar defines the restrictions on how to form a valid sequence of the TMK language constructs.

Here is how the createSimulation goal, one of its sub-goals, processModel in Figure 2, and a selected *Organizer*, *State*, *Transition* and *Operation* are defined with the TMKL2 language EBNF grammar.

```
Goal( name="create_simulation",
    description="create a simulation based on the model constructed
                                    ,→and the parameters values set and prepare the simulation for
    ,→ running", category="Accomplishment", input_parameters={(user, User), (project,
    Project)}, output_parameters={(simulation, Simulation)}, givens="user != None &&
    project != None && model != None &&
    ,→simulation is None", makes="simulation != None",
    means={(process_model, (user, project, model)),
    ,→(prepare_simulation, (simulation))}); mechanisms = {process_model_org,
prepare_simulation_org};
Organizer( name="process_model_org", description="Process the model by setting
    the simulation
    ,→ instance's tick limit to be the input tick limit, process the parameters, ecology nodes
    ,→ and ecology edges of the model by converting them into populations, agent types,
    ,→ simulation parameters, simulation initialization behaviors, tick behaviors, and growth
    ,→ procedures",
    ,→
    input_parameters={(user, User),(project, Project)},
    output_parameters={(simulation, Simulation) }, requires="simulation is
    None", provides="simulation != None",
    states={simualtion_is_none_s, simulation_is_not_none_s}, transitions={create_simuluation_t},);
State( name="simualtion_is_none_s",
    description="",
    task_invocation=(simualtion_is_none_op, simulation));
Transition( name = "create_simuluation_t",
    source_state="simualtion_is_none_s",
    target_state="simualtion_is_not_none_s", data_condition="user_exist_s
    && project_exist_s &&
    ,→model_exist_s"
);
Operation( name="simualtion_is_none_op",
    description="has a simulation been created for the specific
    ,→user, project, model", input_parameters={(simualtion_is_none, boolean_literal)},
    output_parameters={}, requires="", provides="");
```

With the TMKL2 grammar defined in EBNF, we can leverage existing tools such as ANTLR 4[4](Parr et al., 2014) to generate a parser to automatically verify the syntax of the TMK model.

After verifying the syntax of the TMK model, we added logic to the parser to perform static semantic analysis on the following:

1. All references resolve to a specific identifier.
2. The Given condition of each Goal should imply the Requires condition for its subordinateMechanics.
3. The Provides condition for any Mechanism should imply the Makes condition for its superordinate Goal.
4. Each OutputParameter for an Operation should name an Instance in the Environment or a Property of an Instance.
5. FormalArguments correspond in Type to ActualArgument expressions.

We use the .tmk file format to differentiate the formalized TMK model in the EBNF grammar and the .JSON file extension for the equivalent of that TMK model expressed in JSON format. Figure 4 describes the input and output in the Task-Method-Knowledge Modeler Platform. Users can draft their initial TMK model using the TMK Modeler Platform GUI [5] or a large language model. Then, they can convert their TMK model in .JSON to a .tmk or vice versa on the Platform. The ANTLR parser verifies the syntax of the TMK model and performs the static semantic analysis using the .tmk file format. The parser displays the result to the user via the modeler platform. The TMK model is either valid or the line number with parser exception will be displayed.

6. **An Illustrative Example**

We will use a *How?* question posted to VERA, “How does VERA create a simulation?”, as a simple walk-through example. First, we express a self-model of VERA using the formalized TMKL2 grammar. The VERA self-model includes the top-level goal, sub-goals, mechanism to accomplish those goals, and the knowledge that VERA needs to operate.

Once we verify the syntax of the VERA self-model in the .tmk file format, the verified VERA self-model is converted into the equivalent .JSON file format and placed on the MCM-TMK server. MCM-TMK loads the VERA self-model into working memory as a key-value pair dictionary when the MCM-TMK server starts. For the question, "*How does VERA create a simulation?*", MCM-MTK searches for the keys in all dictionaries by semantically matching the question with the keys. Here is the knowledge trace showing the top *k*=4 keys with highest similarity relevance score found in *Task*:

[4] . ANTLR (ANother Tool for Language Recognition) https://www.antlr.org/

[5] . https://arkb0.github.io/Ontology-Modeller-GUI/

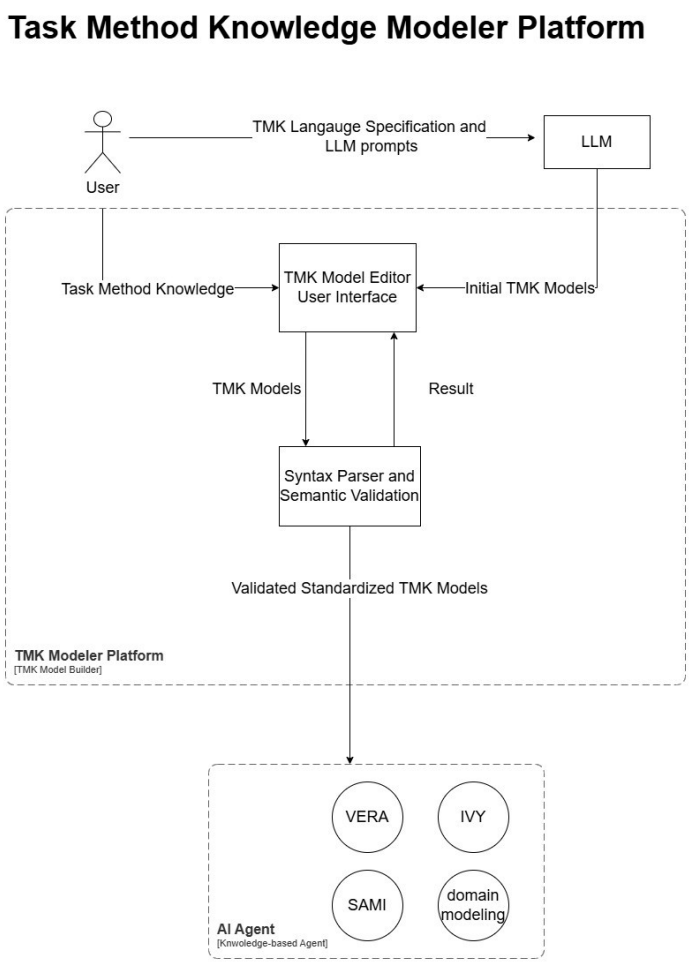


Figure 4. The data flow in the Task Method Knowledge Modeler Platform

[(Document(page_content="{'name': 'create_simulation',
↪ 'description': 'create a simulation based on the model constructed and the parameters
↪ values set and',
↪ 'inputParameters': [], 'outputParameters': [], 'given': 'user created a project and the
↪ project has a model and simulation has not already been created', 'makes': 'simulation
↪ instance created and ????', 'means': []}", metadata={'source': 'Task',
↪ 'seq_num': 2, 'entry': 'create_simulation'}),
↪
0.7584252646630167),
(Document(page_content="{'name': 'run_simulation', 'description':

'start and keep running the simulation that was created based on the model constructed and parameters values set', 'inputParameters': [], 'outputParameters': [], 'given': 'simulation instance is created and number of simulation instance tick executed is less than its tick limit ', 'makes': 'simulation is running', 'means': []}", metadata={'source': 'Task', 'seq_num': 4, 'entry': 'run_simulation'}),
0.719666474863368),
(Document(page_content='{\'name\': \'finish_simulation\', \'description\': "finish running a simulation that\'s created based on the model constructed and parameters values set", \'inputParameters\': [], \'outputParameters\': [], \'given\': \'\', \'makes\': \'\', \'means\': []}', metadata={'source': 'Task', 'seq_num': 1, 'entry': 'finish_simulation'}),
0.7137002723552871),
(Document(page_content='{\'name\': \'prepare_simulation\', \'description\': \'prepares the simulation instance for running by setting its tick executed to 0 and all populations to their initial count\', \'inputParameters\': [], \'outputParameters\': [], \'given\': \'a project is created, simulation instance created\', \'makes\': "simulation\'s number of ticks executed becomes equal to 0", \'means\': []}', metadata={'source': 'Task', 'seq_num': 5, 'entry': 'prepare_simulation'}),
0.7040679473029103)]

The *Task* "create_simulation" has the highest similarity relevance score, 0.758, with respect to the question. The MCM-TMK continues its search using the keys found in "create_simulation". The below knowledge trace shows the top *k*=4 keys found in *Method*:

[(Document(page_content="{'name': 'create_simulation', 'description': '', 'inputParameters': [], 'outputParameters': [], 'requires': 'user created a project and the project has a model and simulation has not already been created', 'provides': 'simulation instance created'}", metadata={'source': 'Method', 'seq_num': 2, 'entry': 'create_simulation'}),
0.7576801083813441),
(Document(page_content="{'name': 'process_model', 'description': '', 'inputParameters': [], 'outputParameters': [], 'requires': 'a project created by the user,project has a model', 'provides': 'simulation instance created'}", metadata={'source': 'Method', 'seq_num': 4, 'entry': 'process_model'}),
0.7140486792785024),
(Document(page_content="{'name': 'prepare_simulation',

'description': '', 'inputParameters': [], 'outputParameters': [], 'requires': 'simulation_instance_created', 'provides': 'the population instance number of current count becomes equal to population instance initial count for population in simulation instance populations'}", metadata={'source': 'Method', 'seq_num': 5, 'entry': 'prepare_simulation'}), 0.6861337678385245),
(Document(page_content="{'name': 'run_simulation', 'description': '', 'inputParameters': [], 'outputParameters': [], 'requires': 'number of simulation ticks executed becomes less than simulation tick limit', 'provides': 'simulation is running'}", metadata={'source': 'Method', 'seq_num': 3, 'entry': 'run_simulation'}), 0.6646906314279399)]

The top *k*=4 documents with highest similarity relevance score are documents 1) "create_simulation" found in *Task* and *Method*, 2)"process_model" and 3)"prepare_simulation" found in *Method*. The large language model used these 4 documents to generate and refine the final answer: "*VERA creates a simulation by taking the ecological model you have built in your project along with the parameter values you have set, and then generating a simulation instance from these. This simulation instance is a ready-to-run version of your model that lets you explore how the ecological system behaves with those settings. The simulation is created once you have a project with a model and no simulation exists yet for it.*." Figure 5 shows the sequence of a user's inquiry, "*How does VERA create a simulation?*", stepping through the VERA metacognitive self-explanation layer and having an answer returned.

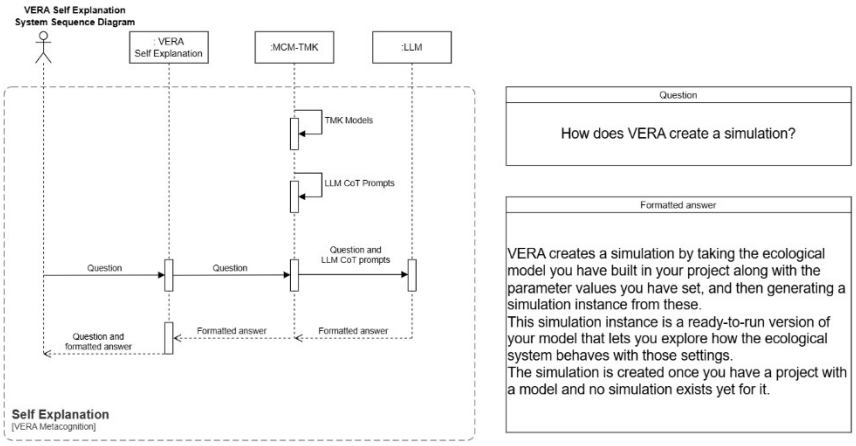


Figure 5. The sequence diagram for the *"How does VERA create a simulation?"* question and answer.

7. **Summary of Empirical Results**

We have elsewhere described empirical results on self-explanation in VERA in detail (Sushri et al., 2024; Goel et al., 2026). Since the focus of this paper is on formalization of TMK models for

automatic verification, here only summarize the empirical results. In Table 5 below, the Recall metric measures the proportion of relevant information retrieved by the self-explanation method compared to the total relevant information available within its knowledge base, and the Accuracy metric assessed the factual correctness of the information presented in the explanations.

Table 5. Summary of empirical results on self-explanation from the VERA experiment

Evaluation	VERA
Dataset	Same 66 questions from the XAI question bank (Liao et al., 2020) as in the SAMI experiments, this time adapted for VERA
Metrics	Recall, Accuracy
Accuracy result	High accuracy on 65/66 questions
Recall result	High recall on virtually all questions across question categories (*What? Why? How?)*

8. **Conclusion**

Despite the advent of GenAI, KBAI remains an important paradigm for research on AI. As usual, a major open question in KBAI is how to represent knowledge. Handcrafting the knowledge representations such as TMK models raises questions about the consistency, generality, reproducibility, and scalability of the KBAI methods. In this paper, we presented a formalization of the TMKL2 grammar in EBNF, a TMK Modeler Platform, a metacognitive architecture for self-explanation, and an algorithm for generating self-explanations. The TMK Modeler Platform partially automates the verification of a handcrafted TMK model of a knowledge-based AI agent and thus makes model verification both more accurate and much easier. Although we used self-explanation in VERA for illustration, the TMKL2 language, the TMK Modeler Platform, the metacognitive architecture, and the algorithm for self-explanation are domain-agnostic and general-purpose. We note that while the TMK Modeler Platform automates the syntactic verification of the TMK models, including the construction of the abstract syntax tree and a static semantic analyzer, it does not automate verification of the operational semantics of the TMK representations. The latter still requires development of the metacognitive architecture and algorithms, and validation of the answers to a user's questions as we briefly illustrated in Section 6.

In a parallel line of work, we are developing a Text-to-Model technique that uses Generative AI to automatically extract a TMK model of an AI skill from natural language descriptions of the skill (Dass et al., 2026). We use the same TMKL2 language in this complementary line of work. The EBNF formalization of TMKL2 helps to maintain consistency and ensure generality across the projects. In future work, we expect to integrate the two projects and provide one general-purpose method for automatic construction and verification of TMK models.

Acknowledgements

We thank members of the VERA and Ivy teams in the Design Intelligence Lab (DILab; dilab.gatech.edu) for their contributions to this research. This research is supported by a US National Science Foundation grant (Grant No. 2247790) to the National AI Institute for Adult Learning and Online Education (AI-ALOE; aialoe.org).

References

An, S., Bates, R., Hammock, J., Rugaber, S., Weigel, E., & Goel, A. (2020). Scientific modeling using large scale knowledge. *Artificial Intelligence in Education* (pp. 20–24). Cham: Springer International Publishing.

An, S., Hammock, J., & Goel, A. (2025). How online learners engage in self-directed modeling: A behavioral analysis. *International Journal of Artificial Intelligence in Education*, *35*, 3501–3528. Published online July 25, 2025.

Backus, J. W. (1959). The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM conference. *Proceedings of the International Conference on Information Processing* (pp. 125–132). UNESCO.

Backus, J. W., et al. (1963). Revised report on the algorithmic language ALGOL 60. *Communications of the ACM*, *6*, 1–17.

Basappa, R., Tekman, M., Lu, H., Faught, B., Kakar, S., & Goel, A. K. (2024). Social ai agents too need to explain themselves. *Generative Intelligence and Intelligent Tutoring Systems* (pp. 351–360). Cham: Springer Nature Switzerland.

Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.

Chandrasekaran, B., & Swartout, W. (1991). Explanations in knowledge systems: the role of explicit representation of design knowledge. *IEEE Expert: Intelligent Systems and Their Applications*, *6*, 47–49.

Chandrasekaran, B., Tanner, M. C., & Josephson, J. R. (1989). Explaining control strategies in problem solving. *IEEE Expert: Intelligent Systems and Their Applications*, *4*, 9–15, 19–24.

Chase, H. (2022). Langchain. GitHub repository.

Cox, M., Alavi, Z., Dannenhauer, D., Eyorokon, V., Munoz-Avila, H., & Perlis, D. (2016). MIDCA: A metacognitive, integrated dual-cycle architecture for self-regulated autonomy. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Cox, M. T., & Raja, A. (Eds.). (2011). *Metareasoning: Thinking about thinking*. Cambridge, MA: The MIT Press.

Dass, R., Puri, S., Khandelwal, A., Jin, X., & Goel, A. K. (2026). Developing models of procedural skills using an AI-assisted text-to-model approach. *Proceedings of the 13th ACM Conference on Learning @ Scale (L@S '26)* (pp. 69–78).

Erol, K., Hendler, J., & Nau, D. S. (1994). HTN planning: complexity and expressivity. *Proceedings of the Twelfth AAAI National Conference on Artificial Intelligence* (p. 1123–1128). AAAI Press.

Gajos, K. Z., & Mamykina, L. (2022). Do people engage cognitively with ai? impact of ai assistance on incidental learning. *Proceedings of the 27th International Conference on Intelligent User Interfaces (IUI '22)* (pp. 794–806). New York, NY, USA: Association for Computing Machinery.

Goel, A., Sushri, S., Alico, N., Luu, J., Khan, R., & Rugaber, S. (2026). Generative AI for metacognition in AI agents. In M. Mayrath, J. Behrens, & D. Robinson (Eds.), *Handbook on generative ai in education*. Springer. In press.

Goel, A. K., & Rugaber, S. (2017). Gaia: A cad-like environment for designing game-playing agents. *IEEE Intelligent Systems*, *32*, 60–67.

Hoang, H., Urban, S., & Avila, H. M. (2005). Hierarchical plan representations for encoding strategic game ai. *Proceedings of the First AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* (p. 63–68). AAAI Press.

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*, 535–547.

Kakar, S., Basappa, R., Camacho, I., Griswold, C., Houk, A., Leung, C., Tekman, M., Westervelt, P., Wang, Q., & Goel, A.K. (2024) SAMI: An AI Actor for Fostering Social Interactions in Online Classrooms. In Proceedings of the *20th International Conference on Intelligent Tutoring Systems*, Spring 2024.

Lee, J., Nandan, V., Sikka, H., Rugaber, S., & Goel, A. (2023). Designing a communication bridge between communities: Participatory design for a question-answering AI agent. *arXiv:2308.00813*.

Liao, Q. V., Gruen, D., & Miller, S. (2020). Questioning the ai: Informing design practices for explainable ai user experiences. *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (pp. 1–15). New York, NY, USA: Association for Computing Machinery.

Murdock, J. W., & Goel, A. K. (2008). Meta-case-based reasoning: self-improvement through self-understanding. *J. Exp. Theor. Artif. Intell.*, *20*, 1–36.

Nau, D. S., Au, T. C., Ilghami, O., Kuter, U., Murdock, J. W., Wu, D., & Yaman, F. (2003). Shop2: An HTN planning system. *Journal of Artificial Intelligence Research*, *20*, 379–404.

Parr, T., Harwell, S., & Fisher, K. (2014). Adaptive LL(*) parsing: The power of dynamic analysis. *SIGPLAN Not.*, *49*, 579–598.

Robson, R., et al. (2022). Intelligent links: Ai-supported connections between employers and colleges. *AI Magazine*, *43*, 75–82.

Rugaber, S. (2011). *Tmkl2: A teleological language for adapting software*. Technical report, DILab, Georgia Institute of Technology.

Schmidt, D. C. (2006). Model-driven engineering. *IEEE Computer*, *39*, 25–31.

Scowen, R. S. (1996). *ISO/IEC 14977:1996 information technology — syntactic metalanguage — extended BNF*. Standard ISO/IEC 14977:1996, International Organization for Standardization.

Sushri, S., Dass, R., Basappa, R., Lu, H., & Goel, A. K. (2024). Combining cognitive and generative ai for self-explanation in interactive ai agents. *HEXED-L3MNGET Workshop, 17th International Conference on Educational Data Mining (EDM '24)*. CEUR-WS.org.

Swartout, W. R. (1983). XPLAIN: A system for creating and explaining expert consulting programs. *Artificial Intelligence*, *21*, 285–325.

Wikipedia (2026). Extended Backus–Naur form — Wikipedia, The Free Encyclopedia. Online; accessed 21-August-2026.

Appendix A. GitHub Repository

Readers can find the full version of VERA pictorial TMK diagram, TMKL2 EBNF grammar definition, the formalized VERA TMK model, and the knowledge trace of the toy example in Section 6 on the GitHub repository https://github.com/aialoe/TMKL-Formalization